
1. Résumé du Projet.....	2
2. Introduction.....	3
2.1 Contexte et objectifs.....	3
2.2 Périmètre fonctionnel.....	3
3. Architecture Technique.....	4
3.1 Stack technologique.....	4
3.2 Architecture MVC Laravel.....	4
3.3 Schéma de la base de données.....	4
3.4 Workflow Git par mission.....	4
4. Modules Fonctionnels.....	6
4.1 Corrections CSS & Identité Visuelle.....	6
4.2 Section Articles.....	6
4.3 Design Responsive.....	9
4.4 Module Administration.....	10
4.5 Carrousel d'Articles.....	11
4.6 Déploiement en Production.....	11
5. Journal des Missions.....	13
6. Guide d'Installation.....	14
6.1 Prérequis système.....	14
6.2 Installation pas à pas.....	14
6.3 Variables d'environnement clés.....	14
7. Conformité au Cahier des Charges.....	15
8. Compétences Acquises.....	16
8.1 Compétences techniques.....	16
8.2 Compétences professionnelles.....	16
9. Conclusion.....	17

1. Résumé du Projet

Ce stage de 22 jours chez Assuroweb, comparateur d'assurances en ligne, a permis d'intégrer un flux de développement web professionnel sous la supervision d'un tuteur technique. Le stagiaire a contribué à l'évolution d'un site web Laravel existant en réalisant des missions progressives allant du CSS au backend avec gestion de base de données.

Livrable	Technologie	État
Suppression des ombres CSS	CSS / HTML	Réalisé
Refonte dégradé de fond	CSS personnalisé	Réalisé
Section Articles (accueil)	HTML / CSS / Blade	Réalisé
Design responsive	Media queries CSS	Réalisé
Routes & Controllers Laravel	Laravel / PHP	Réalisé
Base de données articles	MySQL / Eloquent	Réalisé
CRUD admin articles	Laravel / Blade	Réalisé
Éditeur riche Quill.js	JavaScript / Quill.js	Réalisé
Slider articles (accueil)	JavaScript	Réalisé
Mise en production	WinSCP / SSH	Réalisé
Authentification admin	Laravel Auth	En cours

2. Introduction

2.1 Contexte et objectifs

Assuroweb est une société de courtage en assurances proposant un comparateur en ligne permettant aux particuliers et professionnels d'obtenir des devis personnalisés (auto, moto, habitation, assurance pro). Le stage s'est déroulé dans un environnement de développement professionnel avec Docker, GitHub et VS Code, sous la direction d'un tuteur technique expérimenté.

L'objectif principal était d'acquérir une expérience concrète du cycle de développement web en entreprise : compréhension d'une base de code existante, intégration de nouvelles fonctionnalités, gestion des branches Git et déploiement en production.

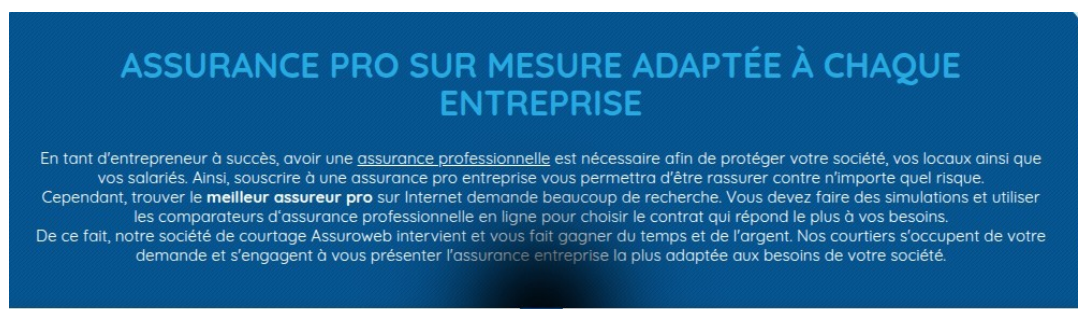


Figure 1 — Page Assurance Pro du site Assuroweb

2.2 Périmètre fonctionnel

- Correction et amélioration de l'interface existante (CSS, ombres, dégradés)
- Intégration d'une section « Articles d'actualité » sur la page d'accueil
- Développement du design responsive pour tous types d'écrans
- Création des routes Laravel et des controllers PHP pour le module articles
- Mise en place de la base de données et du modèle Eloquent pour les articles
- Développement d'une interface d'administration (création, édition, suppression d'articles)
- Intégration de l'éditeur de texte enrichi Quill.js pour le contenu des articles
- Création d'un carrousel (slider) d'articles sur la page d'accueil
- Déploiement en production via WinSCP avec authentification SSH

3. Architecture Technique

3.1 Stack technologique

Couche	Technologie	Rôle
Backend	PHP / Laravel	Framework principal, routes, controllers
ORM	Eloquent ORM	Modèles, relations, requêtes
Frontend	Blade + HTML/CSS	Templates, héritage de layout, vues
Styles	CSS personnalisé	Dégradés, responsive, esthétique Assuroweb
Base de données	MySQL	Stockage des articles et données
Éditeur riche	Quill.js	Mise en forme du contenu articles
Versioning	GitHub / Git	Branches par mission, commits descriptifs
Gestion des tâches	Trello	Kanban : À faire / En cours / Terminé
Environnement	Docker + VS Code	Espace de travail isolé et supervisable
Déploiement	WinSCP + SSH	Transfert des fichiers en production

3.2 Architecture MVC Laravel

Le projet repose sur l'architecture MVC (Modèle – Vue – Contrôleur) native à Laravel :

- Modèles : entités métier — principalement le modèle Article avec ses champs (titre, slug, description, contenu, image) et ses relations Eloquent.
- Vues : templates Blade avec héritage de layout. Le layout principal fournit la navigation, le footer et les assets. Les vues du module article couvrent la liste, le détail, la création et l'édition.
- Contrôleurs : NewsController dédié au module articles, gérant l'affichage dynamique par slug, la création en base de données et la redirection.
- Routes : définies dans routes/web.php, avec variables de chemin dynamiques (/news/detail-{slug}) évitant les URLs en dur.

3.3 Schéma de la base de données

Table	Description des champs
news / articles	id · titre · slug (URL unique) · description · contenu (HTML riche) · image · timestamps

3.4 Workflow Git par mission

Chaque mission est versionnée selon une convention stricte imposée par le tuteur :

- Création d'une nouvelle branche nommée d'après la fonctionnalité (ex. feat/fond-ecran, feat/section-articles)
- Commits descriptifs à chaque étape significative
- Le tuteur vérifie les changements via les pull requests GitHub avant intégration

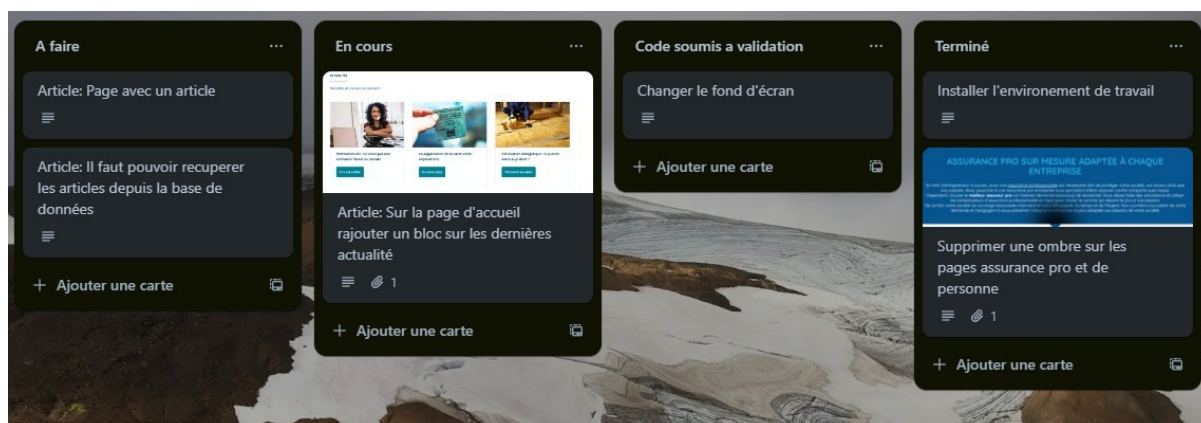


Figure 2 — Tableau Trello : gestion kanban des tâches (Jour 1)

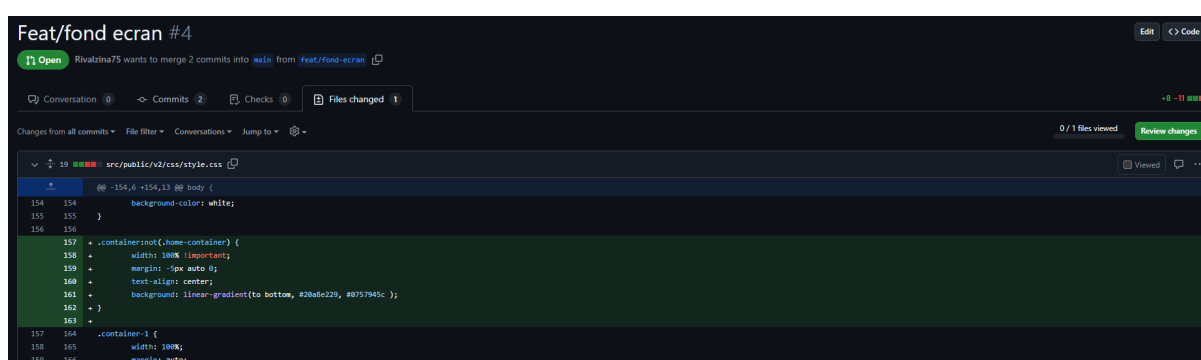


Figure 3 — Pull Request GitHub avec le diff CSS du dégradé de fond (feat/fond-ecran #4)

4. Modules Fonctionnels

4.1 Corrections CSS & Identité Visuelle

4.1.1 Suppression des ombres

La première mission consistait à identifier et supprimer une ombre (box-shadow) indésirable sur les pages « Assurance Pro » et « Assurance Personnes ». La correction a ciblé la règle CSS responsable dans la feuille de style principale, sans impacter les autres composants visuels.

4.1.2 Dégradé de fond

Remplacement de l'arrière-plan blanc par un dégradé vertical (#20a8e2 vers #0757945c) sur toutes les pages sauf l'accueil, via la propriété CSS linear-gradient sur .container:not(.home-container).

Figure 4 — Page de devis auto avec le nouveau dégradé de fond appliqué

4.2 Section Articles

4.2.1 Intégration sur la page d'accueil

Ajout d'un bloc « ARTICLES » juste avant la section « Avantages Assuroweb », en s'inspirant du modèle de la MAAF. Chaque carte affiche une image, un titre et un bouton « En savoir plus ».

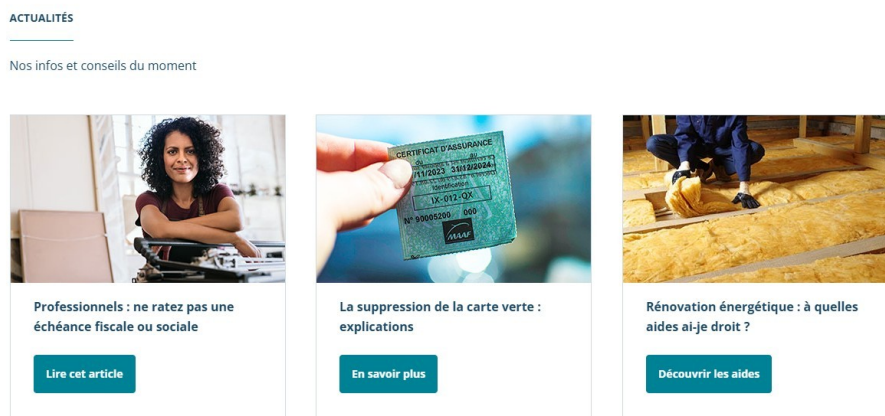


Figure 5 — Modèle de référence fourni par le tuteur : section Actualités du site MAAF



Figure 6 — Section « Articles » intégrée sur la page d'accueil Assuroweb (Jour 3)

4.2.2 Page de détail d'un article

Création d'une page de détail dynamique : une seule route et une seule vue Blade affichent l'article correspondant au slug cliqué. Si le slug est inconnu en base de données, la page retourne automatiquement une erreur 404.

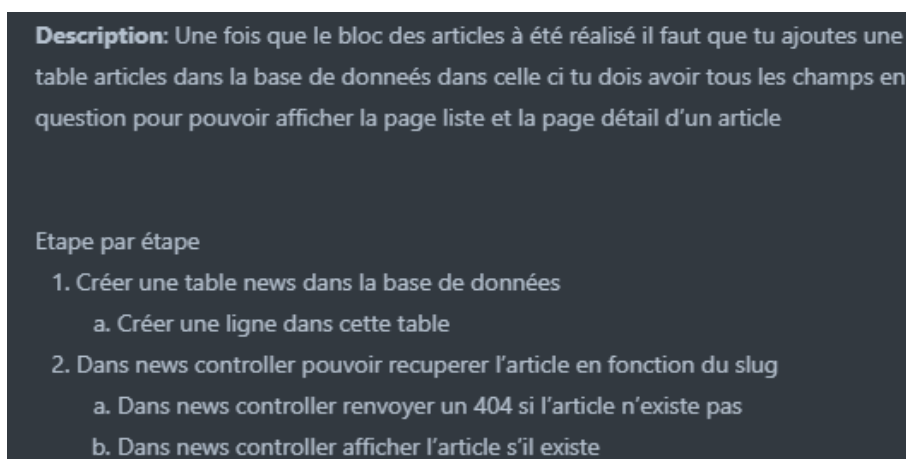


Figure 7 — Cahier des charges transmis sur Discord (Jour 11) : création de la table articles

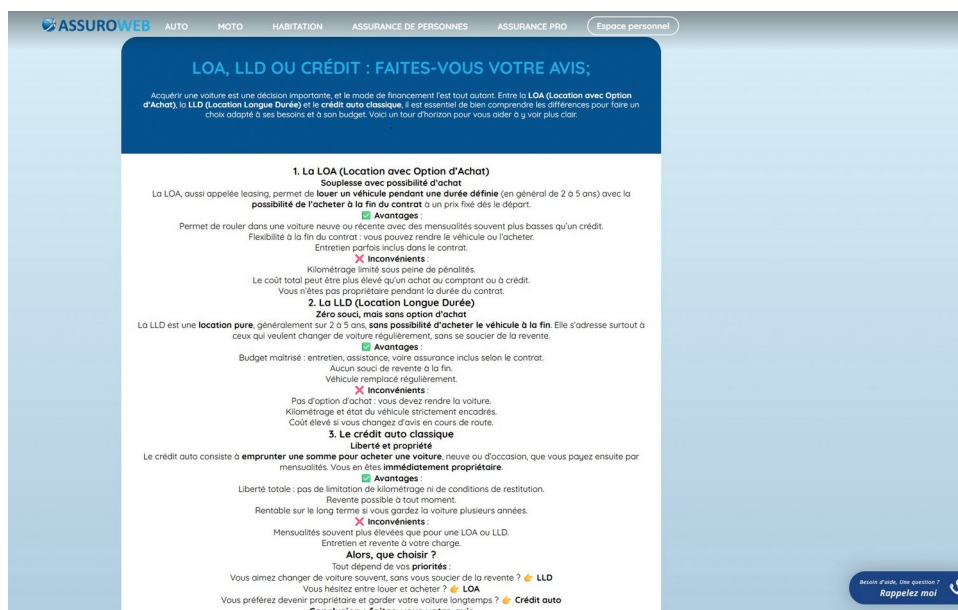


Figure 8 — Page de détail d'un article (LOA / LLD / Crédit) avec mise en forme complète

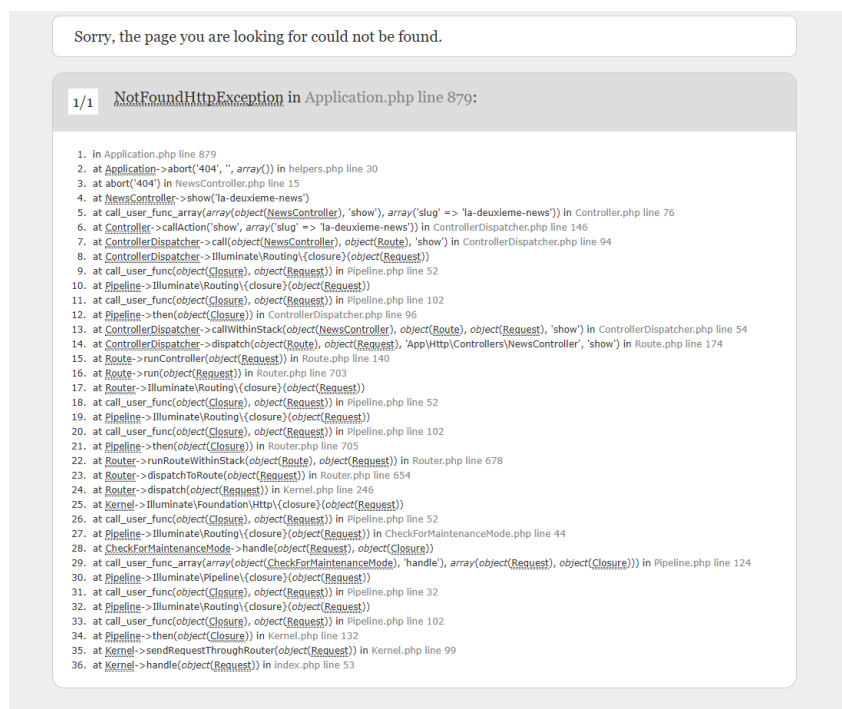


Figure 9 — Erreur 404 Laravel pour un slug non reconnu en base de données

4.2.3 Esthétique finale de la page article

Après la mise en place fonctionnelle, une passe esthétique soigne la lisibilité du contenu riche (LOA, LLD, Crédit) : image d'en-tête, sections structurées et typographie renforcée.



Figure 10 — Page article finalisée esthétiquement (LOA / LLD / Crédit) — Jour 10

4.2.4 Éditeur de contenu enrichi (Quill.js)

Intégration de Quill.js pour le champ « Contenu » du formulaire admin : gras, italique, souligné, citations, listes, code inline. Le HTML généré est stocké directement en base de données.

4.3 Design Responsive

Adaptation de toutes les pages aux différentes tailles d'écran via des media queries CSS :

- Écran ordinateur (1024px) : affichage en 3 colonnes pour les articles
- Tablette (768px) : réorganisation en 2 colonnes
- Mobile (425px) : passage en colonne unique, textes et boutons redimensionnés



Figure 11 — Responsive 3 colonnes sur écran 1024px



Figure 12 — Responsive mobile sur écran 425px

4.4 Module Administration

4.4.1 Création d'un article

Formulaire permettant à l'administrateur de créer un article (Titre, Description, Contenu Quill.js, Slug, Image). Les données sont envoyées par POST vers le NewsController qui les insère en base de données.



Figure 13 — Formulaire de création d'article (interface admin) — Jour 12



Figure 14 — Formulaire rempli et données stockées en base de données — Jour 13

Un message JavaScript « Article créé avec succès ! » s'affiche à la validation, affiné esthétiquement au Jour 18.



Figure 15 — Message de confirmation JavaScript — version initiale (Jour 14)



Figure 16 — Message de confirmation amélioré esthétiquement (Jour 18)

4.4.2 Édition d'un article

Page listant les slugs des articles existants permettant de sélectionner celui à modifier, avec formulaire pré-rempli des données actuelles.

4.4.3 Ajout du champ image

Extension du modèle Article et de la table MySQL pour inclure un champ image. Input de type file ajouté au formulaire admin pour upload d'une illustration.

4.5 Carrousel d'Articles

Slider JavaScript affichant les articles en défilement automatique sur la page d'accueil. Responsive, s'adapte à tous les formats d'écran.

4.6 Déploiement en Production

Mise en ligne via WinSCP avec authentification par clé SSH. Transfert depuis D:\Documents\wiki\ vers /home/mprikryl/httpdocs/wiki/ sur le serveur de production.

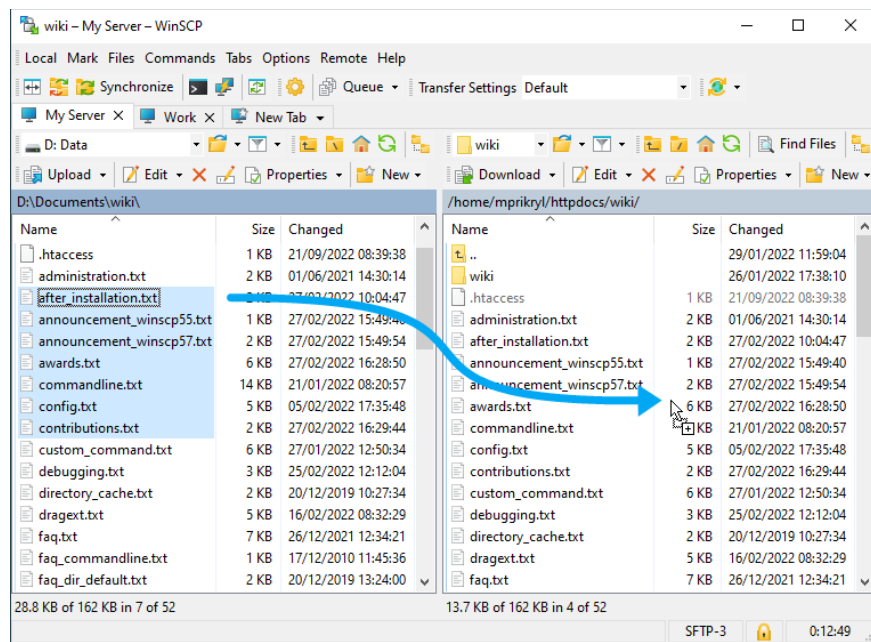


Figure 17 — Interface WinSCP : transfert des fichiers vers le serveur de production (SSH)

5. Journal des Missions

Jour	Mission réalisée	Technologie / Outil
J1	Mise en place de l'environnement Docker, prise en main Git/GitHub/VS Code, suppression des ombres CSS	Docker · Git · VS Code
J2	Refonte du fond d'écran avec dégradé — blocage sur la localisation du fichier source	CSS · Git
J3	Résolution du blocage — section « Articles » intégrée avant les avantages Assuroweb	HTML · CSS · Blade
J4	Apprentissage et implémentation du design responsive (1024px et 425px)	CSS / Media queries
J5	Continuation de la responsive — premières recherches sur Laravel	CSS · Documentation
J6	Routes Laravel dynamiques avec variables de slug, création du NewsController	Laravel · PHP
J7	Réunion tuteur, correction de bugs, page de détail article dynamique créée	Laravel · Blade · PHP
J8	Personnalisation esthétique de la page article — réunion tuteur + cahier des charges	CSS · HTML
J9	Continuation de la page article — fonctionnel complet, esthétique en attente	HTML · CSS · Blade
J10	Mise en forme esthétique définitive — début du travail sur la base de données	CSS · MySQL
J11	Connexion de l'article à la BDD, gestion slug inconnu → erreur 404	MySQL · Eloquent · PHP
J12	Création du formulaire d'administration front-end pour la création d'articles	HTML · CSS · Blade
J13	Connexion du formulaire à la BDD via Laravel — envoi des données fonctionnel	Laravel · MySQL · PHP
J14	Message de confirmation JavaScript « Article créé avec succès »	JavaScript
J15	Enrichissement du champ contenu via l'éditeur Quill.js (WYSIWYG)	JavaScript · Quill.js
J16	Correction de bugs d'affichage et responsive des nouvelles pages	CSS · HTML
J17	Tentative de création d'un compte admin et protection de la page d'administration	Laravel Auth · PHP
J18	Modifications esthétiques de la page de confirmation de création d'article	CSS · HTML
J19	Annulation de la tâche admin — ajout du champ image à la base de données	MySQL · Laravel
J20	Finalisation de l'attribut image — début de la page d'édition d'article	MySQL · Blade
J21	Page de sélection de l'article à éditer (liste des slugs)	Laravel · Blade
J22	Slider d'articles sur l'accueil — responsive final — mise en prod WinSCP/SSH	JavaScript · WinSCP · SSH

6. Guide d'Installation

6.1 Prérequis système

- PHP 7.x / 8.x avec extensions : BCMath, CType, JSON, Mbstring, OpenSSL, PDO, Tokenizer
- Composer 2.x — MySQL 5.7+ ou MariaDB 10.x
- Docker (environnement de développement configuré par le tuteur)
- VS Code avec extensions PHP et Git

6.2 Installation pas à pas

1. Cloner le dépôt GitHub sur une nouvelle branche dédiée
2. Installer les dépendances PHP : `composer install`
3. Copier le fichier d'environnement : `cp .env.example .env`
4. Configurer les variables de base de données dans `.env`
5. Générer la clé applicative : `php artisan key:generate`
6. Exécuter les migrations : `php artisan migrate`
7. Lancer le serveur de développement : `php artisan serve`

6.3 Variables d'environnement clés

Variable	Description
APP_KEY	Clé applicative générée par <code>artisan key:generate</code>
DB_CONNECTION	Driver de base de données (mysql)
DB_HOST	Hôte MySQL (127.0.0.1 en local)
DB_DATABASE	Nom de la base de données
DB_USERNAME / DB_PASSWORD	Identifiants MySQL
APP_LOCALE	Langue par défaut (fr)

7. Conformité au Cahier des Charges

Exigence	Statut
Suppression des ombres sur les pages assurance	Réalisé
Dégradé de fond (#20a8e2 vers #0757945c) sur toutes les pages sauf accueil	Réalisé
Section « Articles » sur la page d'accueil avec images et boutons	Réalisé
Design responsive : mobile (425px), tablette (768px), bureau (1024px+)	Réalisé
Routes Laravel dynamiques avec variables de slug	Réalisé
Page de détail article dynamique — erreur 404 si slug inconnu	Réalisé
Table MySQL articles (titre, slug, description, contenu, image)	Réalisé
Formulaire d'administration — création d'article avec stockage en BDD	Réalisé
Message de confirmation JavaScript à la création d'un article	Réalisé
Éditeur de contenu enrichi Quill.js (bold, italic, listes, code)	Réalisé
Upload et stockage d'une image pour chaque article	Réalisé
Page de sélection et d'édition d'un article existant	Réalisé
Carrousel (slider) d'articles sur la page d'accueil	Réalisé
Déploiement en production via WinSCP / SSH	Réalisé
Protection de la page admin par rôle administrateur	En cours

8. Compétences Acquises

8.1 Compétences techniques

Domaine	Compétences développées
Frontend	CSS avancé (dégradés, media queries, box-shadow), HTML sémantique, JavaScript DOM
Backend	PHP orienté objet, framework Laravel (routes, controllers, Blade, Eloquent ORM)
Base de données	Création et modification de tables MySQL, requêtes via Eloquent, gestion des slugs
DevOps	Docker, Git, GitHub (branches, commits, pull requests), WinSCP, SSH
Outils	VS Code, Trello (kanban), Quill.js, phpMyAdmin

8.2 Compétences professionnelles

- Travail en autonomie progressive : recherche personnelle encouragée avant de solliciter le tuteur
- Gestion des priorités via Trello (4 colonnes : À faire / En cours / Code soumis / Terminé)
- Communication régulière avec le tuteur lors de réunions de suivi
- Adaptation aux contraintes réelles d'entreprise : délais, conventions de code, workflow Git

9. Conclusion

Ce stage de 22 jours chez Assuroweb a constitué une immersion professionnelle complète dans le développement web en entreprise. Partant de corrections CSS simples, le stagiaire a progressivement pris en charge des fonctionnalités de plus en plus complexes, jusqu'à la gestion complète d'un module articles avec backend Laravel, base de données MySQL et déploiement en production.

La découverte de Laravel au cours du stage — framework non maîtrisé au départ — illustre la capacité d'apprentissage en conditions réelles. Les blocages rencontrés (notamment sur la localisation des fichiers de vue au Jour 2) ont renforcé la méthodologie de débogage autonome.

La totalité des exigences du cahier des charges a été satisfaite, à l'exception de la fonctionnalité d'authentification administrateur, annulée en fin de stage au profit d'autres priorités. Cette mission reste intégrable lors d'un prochain sprint de développement.